

Backdoor-Based Watermarking in Multi-Client Split Learning

Yao Zhao , Juan Zhao , Zahir Tari , Nasrin Sohrabi , and Fu Xiao , *Senior Member, IEEE*

Abstract—Split learning (SL) partitions a deep neural network (DNN) into client-side and server-side models, with clients sequentially training the client-side model to reduce local computation. As SL adoption increases, protecting the model’s intellectual property to acknowledge clients’ joint contributions becomes critical. Model watermarking is a promising solution. However, existing schemes, designed for centralized training or adapted from collaborative frameworks, are not applicable to SL, especially in multi-client settings. This is because the sequential training nature of SL introduces three unique challenges: (1) server-side training can erase client-inserted watermarks; (2) shared client-side models allow later clients to overwrite earlier watermarks; and (3) malicious clients can deliberately remove legitimate watermarks. This paper presents the first study on watermarking DNNs in multi-client SL, proposing two methods: MarkSplit for benign environments and MarkSplit+ for adversarial settings with malicious clients. Both leverage a novel watermark sample generation technique named Color-Shape-ID. Specifically, MarkSplit jointly trains main-task and watermark samples within a three-tiered structure (mini-local, local, and global rounds), while MarkSplit+ enhances robustness at the cost of increased resource consumption by dynamically adjusting watermark sample counts per client based on watermark detection accuracy. Experiments demonstrate that both methods maintain high fidelity and robustness, achieving $\sim 98\%$ watermark detection accuracy with only a $\sim 3\%$ main-task accuracy drop on average.

Index Terms—Split learning, model watermarking, malicious manipulation, deep neural network.

I. INTRODUCTION

SPLIT learning (SL) facilitates collaborative training of deep neural networks (DNNs) between multiple resource-constrained clients and a powerful server, as illustrated in Fig. 1. In this vanilla multi-client SL framework [1], clients perform

only lightweight computations and share intermediate representations (referred to as smashed data hereafter), instead of private data, with the server [2], [3]. Inspired by this benefit, SL-based methods are increasingly applied in real-world commercial and industrial settings [4], [5], [6], with start-ups such as Owkin adopting SL for healthcare applications¹. Moreover, emerging empirical studies [7], [8], [9], architectural advancements [10], [11], [12], and community initiatives [13], [14], [15] continue to demonstrate SL’s effectiveness, efficiency, and flexibility. SL is especially applicable to edge intelligence environments comprising a few to tens of heterogeneous devices, such as distributed healthcare institutions [16], vehicular edge networks [17], and smart cities [18].

Such increasing adoption and widespread deployment expose SL models to intellectual property (IP) infringement risks [19]. Prior research has shown that SL models can be illicitly leaked during usage [20]. Obviously, protecting the joint ownership of the model in multi-client SL has become a pressing concern, particularly when each client would claim their own contributions to the SL model training. One widely adopted method is model watermarking, which embeds identifiable signatures, e.g., distinct features or intentional backdoors, into the model [21]. These embedded watermarks can later be extracted to assert rightful model ownership during disputes. Existing model watermarking studies [22], [23], [24] primarily focus on centralized training, and some extend to collaborative settings like federated learning (FL) [25], [26], [27]. However, they are inapplicable for SL due to its unique sequential training process. To date, only one study [20] has explored watermarking in SL, but it is limited to the single-client setting. As validated in Section V, a straightforward adaptation of this existing work [20] to multi-client SL scenarios fails to preserve watermark integrity, highlighting the need for a tailored watermarking method for multi-client SL.

Intuitively, in multi-client SL, it may seem feasible for the server to embed watermarks on behalf of all clients. However, relying on the server to embed each client’s watermark assumes that the server is both trustworthy and unbiased—an assumption that does not hold in adversarial settings. Thus, a more robust strategy is to enable clients to embed their own watermarks. This self-embedding can follow either a parameter-based or backdoor-based approach (please refer to Section II-B for details), with the backdoor-based approach being more practical when internal access to the final model is restricted [28]. To this end, our work focuses on backdoor-based watermarking, enabling clients to embed their personalized watermarks

Received 10 March 2026; revised 12 April 2026; accepted 2 July 2026. Date of publication 9 July 2026; date of current version 1 September 2026. This work was supported in part by Shaanxi Province Science Foundation Fund under Grant 2025JC-YBQN-902, in part by Shaanxi Postdoctoral Research Project under Grant 2025BSHSDZZ248, and in part by the Fundamental Research Funds for the Central Universities, CHD under Grant XJ2024004401. (Corresponding author: Juan Zhao.)

Yao Zhao and Fu Xiao are with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210023, China (e-mail: zhaoyao514@gmail; xiaofu@njupt.edu.cn).

Juan Zhao is with the School of Data Science and Artificial Intelligence, Chang’an University, Xi’an 710064, China (e-mail: juan.zhao@chd.edu.cn).

Zahir Tari is with the School of Computing Technologies, RMIT University, Melbourne, VIC 3000, Australia (e-mail: zahir.tari@rmit.edu.au).

Nasrin Sohrabi is with the School of Information Technologies, Deakin University, Geelong, VIC 3220, Australia (e-mail: nasrin.sohrabi@deakin.edu.au).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3712080>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3712080

¹Owkin: <https://www.owkin.com/>

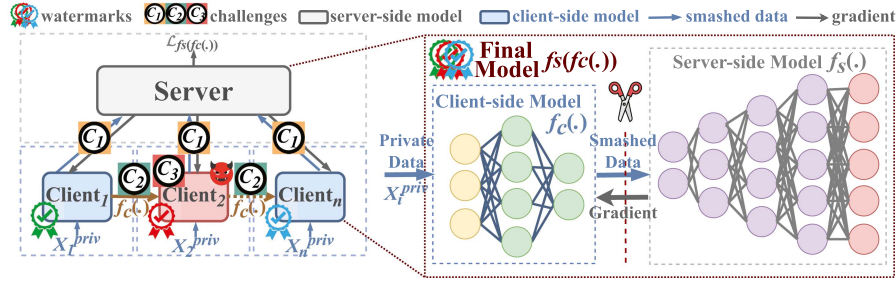


Fig. 1. Watermarking in vanilla multi-client SL. n clients collaboratively train a client-side model $f_c(\cdot)$ using their private data X_i^{priv} , $i \in \{1, 2, \dots, n\}$, while the server trains a server-side model $f_s(\cdot)$. Each client embedding client-specific watermarks into the final model $f_s(f_c(\cdot))$ introduces three challenges that arise at different stages, as elaborated in C_1 , C_2 , and C_3 in Section I.

into the SL model. Such backdoor-based self-embedding can occur at different stages: pre-embedding (before training), mid-embedding (during training), and post-embedding (after training). Pre-embedded watermarks are vulnerable to removal through successive rounds of collaborative training, while post-embedding often severely compromises the model's main task performance [29]. Due to the limitations of both pre- and post-embedding, our focus is on backdoor-based mid-embedding, allowing multiple clients to insert personalized and confidential watermarks during training to claim their joint ownership. Nonetheless, as presented in Fig. 1, implementing this strategy in multi-client SL introduces three challenges, i.e., C_1 , C_2 , and C_3 :

C_1 : how to ensure that all clients' watermark information remains intact throughout server-side training? As clients embed their watermarks during client-side training, the resulting watermark signals are encoded within the smashed data transmitted to the server. These signals should be retained throughout the continued server-side training and effectively reflected in gradient updates sent back to clients.

C_2 : how to ensure that one client's training does not compromise watermarks embedded by others? The client-side model is shared and sequentially updated by clients in collaboration with the server, which introduces the risk that later clients may interfere with watermarks embedded by earlier ones.

C_3 : how to prevent malicious clients from intentionally erasing the watermarks of others? Malicious clients may attempt to overwrite watermarks embedded by others to falsely claim sole ownership of the final model. Due to the lack of access to other clients' private watermarking information, they cannot perform targeted white-box attacks to directly erase embedded watermarks. Moreover, to maintain the final model's utility, adversaries typically avoid aggressive untargeted strategies that could harm main task performance (e.g., fine-tuning the inherited client-side model using unrelated training data). Thus, a more practical and less intrusive strategy for malicious clients is *watermark flooding*, where they inject a disproportionately large number of personal watermark samples during training to dominate watermark signals.

To address these challenges, we design two model watermarking approaches tailored for multi-client SL scenarios: MarkSplit for benign environments and MarkSplit+ for adversarial scenarios with malicious clients. MarkSplit+ extends MarkSplit by enhancing robustness at the cost of increased resource

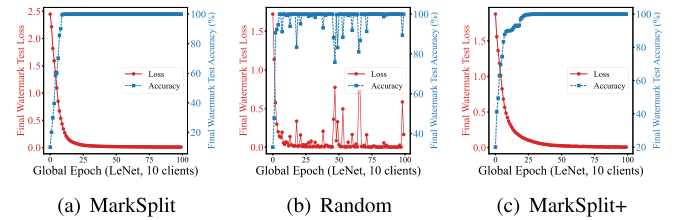


Fig. 2. Sneak peek of methods' performance. Experiments are conducted on CIFAR10 using LeNet with 10 clients, where 20% are malicious under both Random and MarkSplit+ settings. Please see Section V for detailed settings and more results.

consumption. Both methods are applicable in homogeneous and heterogeneous data settings. Their communication efficiency and convergence are theoretically analyzed. Key contributions are summarized as follows:

- We make the first attempt to explore how multiple clients can independently embed personalized and confidential watermarks into SL models. Our investigation reveals that existing watermarking techniques are ineffective in multi-client SL settings because of its inherent sequential training paradigm, where clients perform training one after another. Consequently, watermarks embedded by earlier clients can be easily removed during subsequent server-side training or by the training of the following clients (challenges C_1 and C_2). To handle these challenges, we propose an innovative backdoor-based mid-embedding method named MarkSplit (Section IV-B). MarkSplit jointly trains main task and watermark samples within a unified batch through a three-tiered training structure (i.e., mini-local, local, and global rounds). Moreover, we propose a novel watermark sample generation mechanism called Color-Shape-ID (Section IV-A) to further enhance watermark identifiability, difficulty, scalability, robustness, and stealthiness. A sneak peek of MarkSplit's performance is shown in Fig. 2(a), where it effectively embeds client-specific watermarks with negligible fidelity compromise. See Section V-C for fidelity results.
- We further consider adversarial SL scenarios, where malicious clients launch watermark flooding by injecting an excessive number of their watermark samples to overwrite the watermarks of others (challenge C_3): A straightforward

mitigation strategy is to privately and randomly regulate the number of watermark samples to each client, making it difficult for malicious clients to infer how many watermark samples others are using, thereby reducing the risk of targeted manipulation. However, as shown in Fig. 2(b), this strategy is unstable. Further investigation reveals that adapting the number of watermark samples used in training based on each client's watermark detection accuracy yields better performance. Motivated by this insight, we introduce MarkSplit+ to address challenge C_3 by applying a dynamic watermark allocation strategy (Section IV-C). As shown in Fig. 2(c), MarkSplit+ achieves near-perfect watermark detection accuracy in adversarial scenarios, while ensuring minimal impact on model fidelity, as presented in Section V.

- We comprehensively evaluate MarkSplit and MarkSplit+ across five DNN models (LeNet, AlexNet, ResNet-18, SqueezeNet, and VGG13) and five image datasets (MNIST [30], CIFAR10 [31], CIFAR100 [31], FMNIST [32], and GTSRB [33]) in Section V. Our experiments cover both legitimate and adversarial scenarios, considering homogeneous and heterogeneous data distributions. The evaluation confirms our methods' effectiveness, fidelity, and robustness against five common attacks (fine-tuning, pruning, quantization, backdoor, and poisoning). On average, both methods achieve $\sim 98\%$ watermark detection accuracy with only a $\sim 3\%$ main-task accuracy drop. Our code is available at: <https://anonymous.4open.science/r/Watermarking-8BED/>.

II. RELATED WORK

A. Split Learning

Split learning (SL) is a collaborative learning paradigm that divides a deep neural network (DNN) $\mathcal{F}$ into a client-side model $f_c(\cdot)$ and a server-side model $f_s(\cdot)$, i.e., $\mathcal{F} = f_s(f_c(\cdot))$ [2]. The client trains $f_c(\cdot)$ using private data, while the server handles $f_s(\cdot)$, which carries out the majority of the computational workload. SL adopts a vertically collaborative back-propagation mechanism, in which clients send intermediate representations—referred to as smashed data—to the server for conducting forward and backward propagation of $f_s(\cdot)$ [3], [34], [35]. This design achieves a trade-off among privacy, efficiency, and scalability, making SL highly suitable for privacy-sensitive collaborative AI applications. The optimization problem of SL with n clients is given by

$$\min_{\mathbf{x} \in \mathbb{R}^d} \left\{ \mathcal{F}(\mathbf{x}) := \frac{1}{n} \sum_{i=1}^n \mathcal{F}_i(\mathbf{x}) \right\} \quad (1)$$

where $\mathbf{x}$ is model parameters and $\mathcal{F}_i(\cdot)$ denotes the client-side objective on local dataset $\mathcal{D}_i$ for client c_i , which is defined as

$$\mathcal{F}_i(\mathbf{x}) := \mathbb{E}_{\xi_i \sim \mathcal{D}_i} [f_i(\mathbf{x}; \xi_i)] = \frac{1}{|\mathcal{D}_i|} \sum_{k=1}^{|\mathcal{D}_i|} f_i(\mathbf{x}; \xi_i^k) \quad (2)$$

where $f_i(\cdot)$ is the loss and ξ_i is a data sample randomly chosen from the local dataset $\mathcal{D}_i$.

Technically, SL architectures can be classified into: single-client, multi-client, and U-shape SL [4]. In the single-client setup, the server specifies the model architecture and hyperparameters, partitions the model, and provides the necessary setup. The client manages the initial layers (i.e., $f_c(\cdot)$), while the server handles the remaining layers (i.e., $f_s(\cdot)$). During training, the client processes private data X^{priv} through $f_c(X^{priv})$ and sends the output, i.e., smashed data, to the server, which computes loss, updates model $f_s(\cdot)$, and returns gradients to the client to complete back-propagation on $f_c(\cdot)$. In the multi-client setup (as Fig. 1), clients train collaboratively in a round-robin manner. Each client performs forward and backward passes, updates the client-side model, and hands it off to the next client. The U-shape SL delegates the final model segment to clients to compute the loss locally, preserving label privacy [5]. Our work focuses on multi-client SL settings.

B. Model Watermarking

Model watermarking techniques primarily fall into two categories: parameter-based and backdoor-based approaches [36], [37]. Parameter-based methods encode specific binary strings directly into the model's weights, requiring white-box access for watermark extraction [27], [38]. Backdoor-based methods train the model to produce predefined incorrect outputs when triggered by specific inputs [39]. Since these watermarks can be verified through black-box queries, they are especially useful when internal model access is unavailable.

Watermarking has also been adapted for collaboratively-trained models in federated learning (FL) [25], [29], [37], [40], [41], [42], [43]. Existing FL watermarking strategies can be classified as server-side or client-side watermarking. Server-side watermarking allows the server to embed watermarks on behalf of clients [44]. For example, WAFFLE [29] introduces a retraining phase to inject backdoor-based watermarks into the global FL model. However, these methods often assume a trustworthy server, which may not always be the case, potentially compromising watermark fairness and reliability [40]. Client-side watermarking enables clients to assert ownership directly [45]. Some approaches, e.g., [45], designate a representative client to embed watermarks using gradient-based techniques, while others, like FedIPR [27] and FLClear [43], ensure that every participant can independently embed watermarks and verify their intellectual property rights.

However, existing watermarking solutions designed for FL are incompatible with SL due to fundamental differences in their architectures and threat models. Specifically, in FL, each client trains a complete local model independently, allowing watermarks to be embedded without interfering with others. The FL server only aggregates models without altering them. In contrast, the SL server actively trains its portion of the model, which introduces a critical challenge (C_1): server-side training can overwrite or distort watermarks injected by clients through gradient updates. Additionally, in SL, clients train a shared client-side model sequentially, which creates another key challenge (C_2): subsequent clients may overwrite or corrupt the watermarks embedded by previous ones. In adversarial

scenarios, malicious clients may try to remove the watermarks of others for their own benefit, adding yet another challenge (C_3). These fundamental differences make FL watermarking techniques ineffective in SL, necessitating dedicated designs for SL.

III. THREAT MODEL

We consider an SL system comprising a server and n clients (represented as $\mathcal{C} = \{c_1, c_2, \dots, c_n\}$). The server is only involved with the server-side model architecture and specifies the number of watermark samples each client should use, without accessing clients' specific watermarking details. In our system, all clients can jointly claim ownership of the final SL model ($\mathcal{F} = f_s(f_c(\cdot))$) by black-box verification, instead of just one having exclusive rights, as each contributes valuable private data for model training. Although the server participates in training, it is regarded as part of the system infrastructure and excluded from ownership verification. We focus on backdoor-based self-embedding, where each client independently embeds its unique watermark into the model during training. This ensures that private watermarking information remains confidential and inaccessible to unauthorized parties, including the server. Our solutions aim to achieve three objectives, i.e., O_1 , O_2 , and O_3 :

- *Effectiveness (O_1):* Watermarks must be reliably detectable in final SL models for ownership verification.
- *Fidelity (O_2):* Watermarking processes should have minimal impact on main task performance.
- *Robustness (O_3):* Embedded watermarks should be resistant to various attacks, including those defined in Section III-A.

A. Adversary's Assumption

The adversary could be either malicious clients involved in SL model training or external attackers who obtain the final SL model after training. Malicious clients are assumed to follow the standard SL training process, e.g., contributing all their main task samples, to maximize the model utility and value, as in [29]. They can control their own dataset and the corresponding client-side model. We consider an adaptive adversary who knows our watermark mechanisms and performs adaptive attacks during training via *watermark flooding*, where they inject an excessive number of their own watermark samples, e.g., twice the assigned number used in Section V, to weaken the impact of watermarks embedded by legitimate clients. Further discussion on the number of watermark samples utilized by malicious clients is provided in Section VI. We assume the adversary does not embed other forms of backdoor-based behavior for ensuring model utility. After obtaining a copy of the final model, the adversary tries to remove embedded watermarks by performing the following attacks (implementation details provided in Section V-D). Such assumptions are commonly adopted in prior work [44], [46], [47], [48], [49]. Please note that existing backdoor detection and removal techniques, e.g., [50], [51], are not applicable for removing the watermark embedded by our proposed methods, as they are designed under fundamentally

different system settings, data-access assumptions, and watermark construction mechanisms.

- *Fine-tune:* The adversary re-trains the final model using new main task data samples in an attempt to remove the embedded watermarks.
- *Prune:* The adversary removes certain parts of the final model that potentially contain watermark signals, aiming to weaken the embedded watermarks while ensuring the model's functionality.
- *Quantization:* The adversary reduces the precision of the final model's parameters by representing them with lower-bit numerical formats, e.g., converting 32-bit floating-point values to 16-bit.
- *Backdoor Attack:* The adversary inserts a hidden trigger into the final model to disrupt the embedded watermarks.
- *Data poisoning Attack:* The adversary uses contaminated training samples to corrupt watermark features.

We assume a trusted server. However, the proposed method remains effective even if the server is semi-honest or malicious. Specifically, in the semi-honest setting, inference attacks do not compromise the watermark, since it is embedded locally at the client side and preserved in the final SL model. In the malicious setting, beyond the aforementioned adversarial manipulations, practical utility considerations discourage the server from performing attacks that would substantially degrade overall model performance. Furthermore, from the server's perspective, only smashed data, rather than raw inputs, are accessible, which precludes direct observation of watermark patterns. Moreover, because the smashed data of main-task and watermark samples are statistically indistinguishable due to a lack of a stable reference distribution, anomaly detection mechanisms cannot reliably identify client watermarks.

B. Adversary's Goal

The adversary aims to obtain a model with high performance on the main task and either claim sole ownership or deny any IP involvement, thereby gaining unauthorized benefits, such as academic credit [52]. Ideally, the model should maintain the comparable main task accuracy (or another relevant metric) as the original, although a slight reduction may be acceptable, as in [21]. Additionally, the cost of removing the watermark should not exceed the overall expenses associated with gathering, labeling data, and training a new model.

IV. OUR DESIGN

We present our watermark sample generation method (Section IV-A), and elaborate on the design of MarkSplit (Section IV-B) and MarkSplit+ (Section IV-C). Key notations are listed in Appendix A (available online).

A. Watermark Sample Generation: Color-Shape-ID (CSI)

To watermark an SL model via backdoors, one of the key challenges is to find a set of watermark samples that trigger

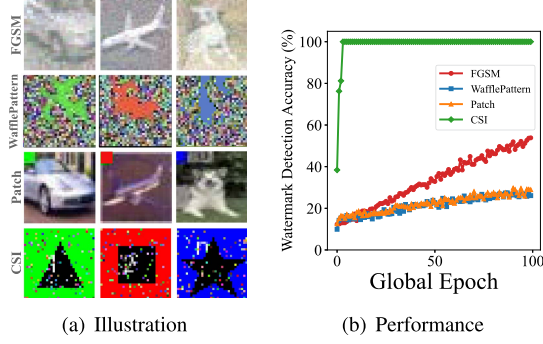


Fig. 3. We evaluate four watermark sample generation mechanisms using LeNet on CIFAR10 in MarkSplit (Section IV-B), involving 5 clients. Other settings are consistent with Section V. The watermark examples are illustrated in Fig. 3(a), and the corresponding watermark detection accuracy is reported in Fig. 3(b).

unique responses only from the intended model, thereby enabling reliable ownership verification. To achieve this, an effective watermarking strategy in multi-client SL should satisfy: **Client-specific Identifiability** - the watermark should encode a client-specific signature to distinguish different clients; **Difficulty** - the watermark should be difficult enough that only a well-trained model can predict them correctly; **Scalability** - the watermarking strategy must scale with the number of clients; **Robustness** - the watermark should be resistant to adaptive attacks; and **Stealthiness** - the watermark embedded for one client should be hidden from others.

To meet these requirements, we propose a novel watermark sample generation method named Color-Shape-ID (CSI), in which each client's watermark samples are characterized by a unique combination of color, distinct shape, and an embedded identifier, such as the client's ID. The formal specification of CSI is provided as follows. (1) *Encoding Space Definition*: CSI encodes client-specific triggers as triples $w = (c, s, i) \in \mathcal{W} = \mathcal{C} \times \mathcal{S} \times \mathcal{I}$, where $\mathcal{C}$ and $\mathcal{S}$ are sets of colors and shapes, and $\mathcal{I}$ indexes clients; (2) *Rendering Pipeline*: each watermark is rendered by placing shape s on a background with color c , embedding client ID i , and normalizing the image; (3) *Noise Augmentation*: a set of augmented watermarks is generated from each unique watermark via stochastic transformations $\tilde{x}_w = \mathcal{A}(x_w; \theta_{\text{noise}})$, including rotation, translation, scaling, color jitter, and Gaussian noise to enhance robustness; (4) *Collision Analysis*: the total number of unique watermarks is $|\mathcal{W}| = |\mathcal{C}| \cdot |\mathcal{S}| \cdot n$, with collisions only occurring if the number of clients exceeds $|\mathcal{W}|$; (5) *Label Assignment*: each client c_i is assigned a unique label $y_i \in \mathcal{Y}$, which serves as the target output for the final SL model.

We compare our proposed CSI with three existing watermark sample generation mechanisms, including FGSM [53], WafflePattern [54], and Patch [55]. As shown in Fig. 3, CSI demonstrates superior performance by achieving 100% accuracy in the early training rounds.

Rationale Behind CSI: Through validation experiments, we observe that the proposed CSI achieves superior watermark detection accuracy while demonstrating the fastest convergence speed, as illustrated in Fig. 3(b). This improvement is attributed to the incorporation of three orthogonal features—color, shape,

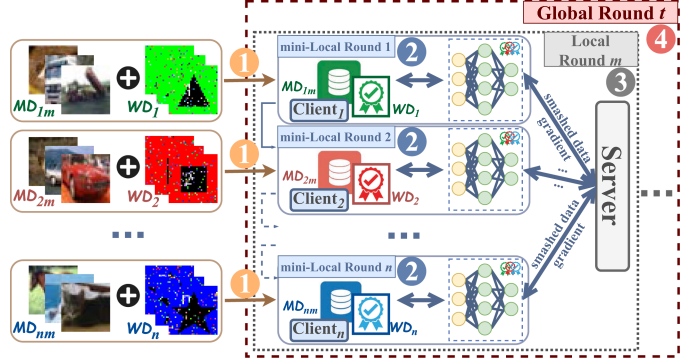


Fig. 4. MarkSplit overview.

and client number—into watermark samples. In addition, noise augmentation improves robustness against removal. This design makes the watermark highly learnable and scalable while keeping it isolated from the main task decision boundary, thereby preserving high fidelity as confirmed in Section V-C. Moreover, CSI satisfies the previously outlined criteria. Specifically, it guarantees **client-specific identifiability** and **scalability** by embedding client-specific scalable identifiers into each watermark sample, enabling clear attribution even in large-scale multi-client environments. It achieves **difficulty** by leveraging a combination of three distinct elements—color, shape, and client identifier—rather than relying on a single feature, thereby increasing the complexity and discrimination of watermarks. Watermarking approaches that incorporate CSI exhibit strong **robustness** against various attacks, as demonstrated in Section V-D. CSI maintains **stealthiness**, since each client embeds its watermark locally and privately without revealing watermark samples to others (including the server). Thus, malicious clients cannot directly observe or infer the watermark patterns of other clients, which significantly limits their ability to perform targeted watermark removal.

B. Legitimate Scenario Solution: MarkSplit

MarkSplit aims to enable clients to independently and gradually embed personal watermark information into the final SL model during training, ensuring equal participation and watermark confidentiality. Given $\mathcal{T}$ global rounds, before training begins, clients and server agree on essential hyperparameters, such as the learning rate, and determine the number of watermark samples k each client will use per local round. The training workflow involves four steps, as illustrated in Fig. 4, summarized in Algorithm 1, and described as follows:

- ① *Dataset Preparation (lines 1–5)*: Suppose $\mathcal{MD}_i$ denotes the main task training dataset for client c_i , with $|\mathcal{MD}_i|$ representing the total number of main task samples. c_i constructs a global training dataset $\mathcal{D}_i = \{\mathcal{MD}_i, \mathcal{WD}_i\}$ by combining $\mathcal{MD}_i$ with a personalized watermark dataset $\mathcal{WD}_i$, which contains k watermark samples. Watermark sample generation method is detailed in Section IV-A. The global dataset $\mathcal{D}_i$ is partitioned into $\mathcal{M} = \frac{|\mathcal{MD}_i|}{B}$ subsets, corresponding to $\mathcal{M}$ local training rounds, where B is batch size. Each subset $\mathcal{D}_{im} \in \mathcal{D}_i$, $m \in \{1, 2, \dots, \mathcal{M}\}$,

Algorithm 1: MarkSplit.

Input: client set $\mathcal{C} = \cup\{c_i\}_{i=1}^n$, initial client-side model $f_c(\cdot)$, initial server-side model $f_s(\cdot)$, batch size $\mathcal{B}$
Output: final client-side model $f_c(\cdot)$, final server-side model $f_s(\cdot)$

```

\\① Dataset Preparation
1: for  $c_i \in \mathcal{C}$  do
2:   for  $m \in \mathcal{M}$  do
3:      $\mathcal{D}_{im} = \{\mathcal{MD}_{im}, \mathcal{WD}_i\}$ 
4:   end for
5: end for
\\④ Global Training
6: while  $t \leq \mathcal{T}$  do
  \\③ Local Training
  7: while  $m \leq \mathcal{M}$  do
    \\② Mini-local Training
    8: for  $c_i \in \mathcal{C}$  do
    9:    $sd_i \leftarrow f_c(\mathcal{D}_{im}), \mathcal{D}_{im} \in \mathcal{D}_i$ 
    10:   $f_s(\cdot) \xleftarrow{\Delta} sd_i$  server-side
    11:   $grad_i \leftarrow f_s(sd_i)$ 
    12:   $f_c(\cdot) \xleftarrow{\Delta} grad_i$ 
    13:   $c_{i+1} \xleftarrow{\Delta} f_c(grad_i)$ 
    14: end for
    15: end while
    16: end while

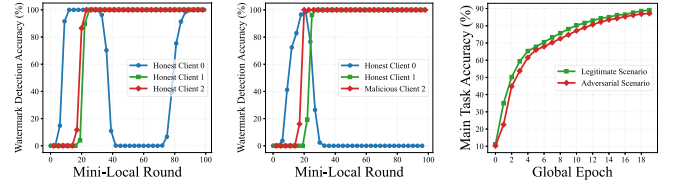
```

Note 1. Case-specific determination of local training rounds $\mathcal{M}$.

Note 2. $\xleftarrow{\Delta}$ represents message transmission.

contains one-batch, i.e., $\mathcal{B}$, main task training samples (i.e., $\mathcal{MD}_{im}$), along with the watermark dataset $\mathcal{WD}_i$. In each local training round m , client c_i uses $\mathcal{D}_{im} = \{\mathcal{MD}_{im}, \mathcal{WD}_i\}$ to train the client-side model $f_c(\cdot)$, ensuring the model repeatedly encounters the same watermark samples to enhance memorization.

- ② *Mini-local Training (lines 8–14)*: A mini-local training round is defined as the execution of one forward-backward training by each client c_i . Specifically, client c_i utilizes $\mathcal{D}_{im}$ to train the latest client-side model $f_c(\cdot)$. The smashed data sd_i is then forwarded to the server to train the server-side model $f_s(\cdot)$. The server computes and returns the corresponding gradients $grad_i$, which are subsequently used to update $f_c(\cdot)$ via backpropagation. After updating $f_c(\cdot)$, client c_i forwards the model to the next client c_{i+1} (if $c_i = c_n$, the model is returned to c_1 to initiate the next round).
- ③ *Local Training (lines 7–15)*: A single local training round is defined as the completion of one mini-local training round by all clients. In homogeneous settings, where all clients have an equal number of main task samples, training naturally concludes when each client's global training dataset is fully utilized. However, in heterogeneous environments, where clients possess varying numbers of samples, some clients may finish earlier than others. To address this imbalance, we adopt a dynamic batching strategy. Specifically, the batch size $\mathcal{B}$ for each client c_i is adaptively adjusted based on the size of its dataset, such that all clients can complete their local training rounds within the same number of local iterations. This ensures clients synchronization while



(a) Legitimate Scenario (b) Adversarial Scenario (c) Main Task Accuracy

Fig. 5. Impact of watermark flooding. The experiment is conducted with LeNet on MNIST. The legitimate scenario involves three honest clients, while the adversarial scenario consists of two honest clients and one malicious client. Please see Section V for detailed settings and more experimental results.

preventing under-utilization or over-training of individual datasets.

- ④ *Global Training (lines 6–16)*: A global training round (i.e., global epoch) is defined as the point at which all clients c_i have processed their entire main task training dataset $\mathcal{MD}_i$, with each sample used only once. Steps ② and ③ are repeated for each global round $t \in \mathcal{T}$.

Rationale Behind MarkSplit: As discussed in Section I, later training can potentially overwrite earlier watermarks. By incorporating mini-local rounds with client-side model handoff, MarkSplit ensures that each client's watermark is continuously preserved and reinforced throughout training. Unlike the standard SL workflow, where the client-side model sees each watermark only once per local training, this design allows the model to repeatedly encounter all clients' watermarks, significantly enhancing its capacity to retain and reliably embed them.

Remark 1: The worst-case communication complexity of MarkSplit is $\mathcal{T} \times \mathcal{M} \times n \times \delta$, where δ denote the total size of the transmitted data.

Proof details of Remark 1 can be found in Appendix B.1 (available online).

C. Adversarial Scenario Solution: MarkSplit+

In legitimate scenarios, all clients are instructed to use the same number of watermark samples k per local round. While this simplifies coordination, it also gives adversaries more flexibility to perform watermark flooding, i.e., using more of their watermark samples during training, to overwrite others' watermarks (challenge C_3). As validated in Fig. 5, when malicious client performs watermark flooding with two times more watermark samples, it degrades both the watermark accuracy of the next client, i.e., Client 0 (Fig. 5(b)), and main task performance (Fig. 5(c)).

To mitigate this issue, we propose MarkSplit+, as illustrated in Fig. 6 and summarized in Algorithm 2, which incorporates a dynamic watermark allocation strategy (as presented in Algorithm 2, lines 15–24). Specifically, for each client c_i , the server adjusts the number of watermark samples $k_i(m+1)$ for the next round $m+1$ based on its current-round watermark detection accuracy $acc_i(m)$. If $acc_i(m) \geq Acc$ (threshold), the default allocation is used, i.e., $k_i(m+1) = WD$ (lines 16–17). If $acc_i(m)$ is slightly below Acc (e.g., within 10%), the allocation is increased to $k_i(m+1) = (1+h) \times WD$ (lines 18–19).

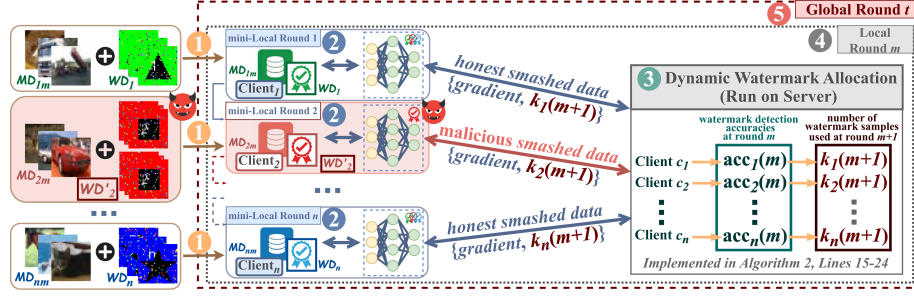


Fig. 6. MarkSplit+ overview. The server dynamically determines the number of watermark samples $k_i(m+1)$ assigned to each client c_i for round $m+1$, based on the client's watermark detection accuracy $acc_i(m)$ in round m .

Algorithm 2: MarkSplit+.

Input: client set $\mathcal{C} = \cup\{c_i\}_{i=1}^n$, initial client-side model $f_c(\cdot)$, initial server-side model $f_s(\cdot)$, batch size $\mathcal{B}$, target watermark detection accuracy Acc , baseline watermark sample number WD , scale factor h

Output: final client-side model $f_c(\cdot)$, final server-side model $f_s(\cdot)$

```

1:  $k_i(0) = WD, i \in \{1, 2, \dots, n\}$ 
2: while  $t \leq \mathcal{T}$  do
3:   while  $m \leq \mathcal{M}$  do
4:     for  $c_i \in \mathcal{C}$  do
5:        $\mathcal{D}_{im} = \{MD_{im}, WD_{ik_i}\}$ 
6:        $sd_i \leftarrow f_c(\mathcal{D}_{im}), \mathcal{D}_{im} \in \mathcal{D}_i$ 
7:        $f_s(\cdot) \leftarrow sd_i$  server-side
8:        $grad_i \leftarrow f_s(sd_i)$ 
9:        $k_i(m+1) \leftarrow \text{WATERMARK\_ADJUSTMENT}(acc_i(m))$ 
10:       $f_c(\cdot) \leftarrow grad_i, k_i(m+1)$ 
11:       $c_{i+1} \leftarrow f_c(grad_i)$ 
12:     end for
13:   end while
14: end while
15: function  $\text{WATERMARK\_ADJUSTMENT}(acc_i(m))$ 
16:   if  $acc_i(m) \geq Acc$  then
17:      $k_i(m+1) \leftarrow WD$ 
18:   else if  $acc_i(m) \geq Acc - 10\%$  then
19:      $k_i(m+1) \leftarrow (1+h) \times WD$ 
20:   else
21:      $k_i(m+1) \leftarrow (1+2 \times h) \times WD$ 
22:   end if
23:   return  $k_i(m+1)$ 
24: end function

```

Note 1. The implementation differences introduced in MarkSplit+ are highlighted in dark red.

If $acc_i(m)$ is significantly below Acc , it is further increased (e.g., $k_i(m+1) = (1+2 \times h) \times WD$) to enhance robustness (lines 20-21). The updated $k_i(m+1)$ is sent to client c_i along with gradient $grad_i$ to guide watermark generation in round $m+1$ (lines 9-10). In our experiments, $Acc = 90\%$ and $h = 0.2$ yield strong performance, though these parameters can be adjusted for different scenarios. We assume the server can accurately

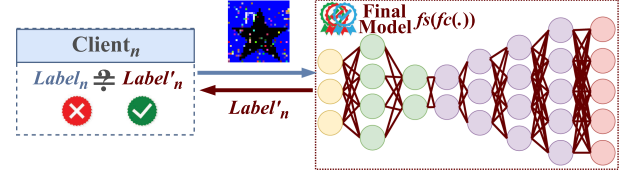


Fig. 7. Ownership verification overview. Each client c_n can use its privately known trigger input to query the model and validate joint ownership based on the returned label.

measure watermark detection accuracy and reliably provide $k_i(m+1)$, which is reasonable as the server is incentivized to maintain overall model utility.

Rationale Behind MarkSplit+: Since the number of watermark samples $k_i(m+1)$ is privately shared between the server and client c_i , it is challenging for malicious clients to accurately estimate the number of samples required to neutralize others' watermarks. Specifically, if malicious clients attempt to amplify the influence in the current local training round by injecting additional watermark samples, the server can respond by directing legitimate clients to increase their own watermark sample counts in the subsequent round. This adaptive strategy allows MarkSplit+ to mitigate watermark flooding while enhancing robustness compared to MarkSplit.

Remark 2: The worst-case communication complexity of MarkSplit+ is $\mathcal{T} \times \mathcal{M} \times n \times \delta'$, where $\delta' = \delta + |k_i(m+1)|$.

Proof details of Remark 2 can be found in Appendix B.2 (available online).

Black-box Ownership Verification: Once training is done by MarkSplit and MarkSplit+, the final model $\mathcal{F} = f_s(f_c(\cdot))$ can be deployed to its intended environment. Each client c_i can then verify and assert *joint ownership* by invoking an API and supplying their personalized watermark samples as inputs. The deployed model processes these inputs and returns a prediction, which is compared against the expected (genuine) label associated with the client's unique watermark. A successful match confirms rightful joint ownership and model integrity, whereas a mismatch indicates potential model tampering or theft. The end-to-end verification workflow is illustrated in Fig. 7.

Remark 3: In MarkSplit and MarkSplit+, the communication overhead increases proportionally with the number of local rounds $\mathcal{M}$ and is inversely related to batch size $\mathcal{B}$.

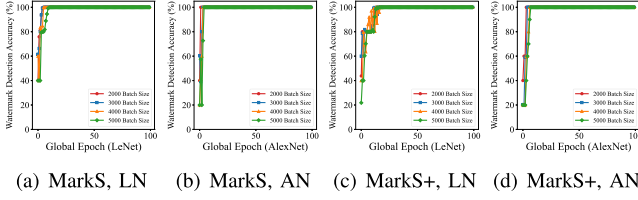


Fig. 8. Impact of batch size $\mathcal{B}$ on MarkSplit (MarkS) and MarkSplit+ (MarkS+). The experiments are conducted using LeNet (LN) and AlexNet (AN) on CIFAR10 with 5 clients. Other settings follow those specified in Section V.

Proof (Remark 3): As shown in Remark 1 and Remark 2, for each client, given a fixed number of local data samples $|\mathcal{MD}|$, the number of local rounds is determined by $\mathcal{M} = \frac{|\mathcal{MD}|}{\mathcal{B}}$. During each local training round $m \in \mathcal{M}$, the client communicates with the server, transmitting data of size δ . Thus, total communication overhead across all rounds becomes $\mathcal{M} \times \delta$. If batch size $\mathcal{B}$ is doubled, the number of local training rounds $\mathcal{M}$ is halved, which directly reduces overall communication overhead by approximately 50%. This concludes the proof.

Remark 3 demonstrates a clear trade-off between batch size $\mathcal{B}$ and communication efficiency. Using a larger $\mathcal{B}$ effectively mitigates communication overhead, making it an important consideration when deploying MarkSplit or MarkSplit+ in resource-constrained environments. However, as demonstrated in Fig. 8, increasing $\mathcal{B}$ slightly reduces convergence speed but maintain watermark detection accuracy. The optimal value of $\mathcal{B}$ depends on specific system requirements. For example, in low-bandwidth environments (e.g., IoT or remote edge devices), where minimizing communication overhead is crucial, a larger $\mathcal{B}$ is preferred to reduce the frequency of communication between clients and the server. Conversely, in scenarios where rapid convergence is more important than communication efficiency, e.g., real-time analytics, a smaller $\mathcal{B}$ could be set to accelerate training with increased communication overhead. The convergence of MarkSplit and MarkSplit+ is analyzed based on the following assumptions.

Assumption 1: Following [56], [57], [58], we assume that

- *L-smoothness:* Each client-side objective function $\mathcal{F}_i(\cdot)$, $i \in \{1, 2, \dots, n\}$, is L -smooth, i.e., its gradient is Lipschitz continuous with constant $L > 0$ such that $\|\nabla \mathcal{F}_i(x) - \nabla \mathcal{F}_i(y)\| \leq L\|x - y\|$, $\forall x, y \in \mathbb{R}^d$.
- *α -strongly convex:* Each client's objective function $\mathcal{F}_i(\cdot)$, $i \in \{1, 2, \dots, n\}$, is α -strongly convex, i.e., there exists a constant $\alpha > 0$ such that $\mathcal{F}_i(y) \geq \mathcal{F}_i(x) + \nabla \mathcal{F}_i(x)^\top (y - x) + \frac{\alpha}{2}\|y - x\|^2$, $\forall x, y \in \mathbb{R}^d$.
- *β^2 -boundness:* The stochastic gradient grad_i at each client c_i has bounded variance, i.e., $\mathbb{E}[\|\nabla f_i(x_i; \xi_i) - \nabla \mathcal{F}_i(x_i)\|^2 | x_i] \leq \beta^2$.
- *δ^2 -heterogeneity:* The level of heterogeneity across clients is quantified by a constant δ^2 such that $\frac{1}{n} \sum_{i=1}^n \|\nabla \mathcal{F}_i(x^*)\|^2 = \delta^2$, where $x^* \in \arg \min_{x \in \mathbb{R}^d} \mathcal{F}(x)$ denotes a global minimizer.

Theorem 1: Given Assumption 1, for both MarkSplit and MarkSplit+, there exists an effective learning rate $\frac{1}{\alpha\mathcal{T}} \leq \tilde{\eta} \leq \frac{1}{6L}$ and weights $w_r = (1 - \frac{\alpha\tilde{\eta}}{2})^{-(t+1)}$, such that the following

holds.

$$\mathbb{E}[\mathcal{F}(\bar{x}^T) - \mathcal{F}(x^*)] \leq 3\alpha\|x^0 - x^*\| \exp\left(-\frac{\alpha\tilde{\eta}\mathcal{T}}{2}\right) + \frac{6\tilde{\eta}\beta^2}{n\mathcal{M}} + \frac{45L\tilde{\eta}^2\beta^2}{4n\mathcal{M}} + \frac{15L\tilde{\eta}^2\delta^2}{2n} \quad (3)$$

Proof (Theorem 1): During each global round $t \in \mathcal{T}$, MarkSplit and MarkSplit+ conduct $\mathcal{M} = \frac{|\mathcal{MD}|}{\mathcal{B}}$ local rounds, where, in each local round $m \in \mathcal{M}$, client c_1 performs client-side model update using a single batch of size $\mathcal{B}$ from its local dataset $\mathcal{MD}_1$, then transmits the updated client-side model to client c_2 , with this sequential updating process continuing iteratively through all clients c_i , $i \in \{1, 2, \dots, n\}$, until each has processed one batch of their local data. Let $\mathbf{x}_i^t$ represent the local parameter at client c_i after its client-side update in global round t , and $\mathbf{x}^t$ denote the global parameter, which combines both client-side and server-side components. Assuming Stochastic Gradient Descent (SGD) is used as the local optimizer, the update rule at client c_i is given by $\mathbf{x}_i^{t+1} = \mathbf{x}_i^t - \eta \mathbf{g}_i^t$, where η is the learning rate, and $\mathbf{g}_i^t := \nabla f_i(\mathbf{x}_i^t; \xi_i)$ denotes the stochastic gradient of $\mathcal{F}_i(\cdot)$ with respect to parameters $\mathbf{x}_i^t$. The initialization is defined as $\mathbf{x}_i^0 = \begin{cases} \mathbf{x}^t, & \text{if } i = 1 \\ \mathbf{x}_i^{t-1}, & \text{if } i > 1 \end{cases}$. The updated global model is denoted by $\mathbf{x}^{t+1} = \mathbf{x}_c^t$.

Analogous to the client drift in FL [57], the client drift in SL is caused by client-side model updates, defined as $E_t := \sum_{i=1}^n \sum_{m=0}^{\mathcal{M}-1} \mathbb{E} \|\mathbf{x}_i^{(t,m)} - \mathbf{x}^t\|^2$. Given $\eta \leq \frac{1}{6Ln\mathcal{M}}$, E_t is bounded by

$$\mathbb{E}[E_t] \leq \frac{9}{2}Ln^3\mathcal{M}^3\eta^2\mathbb{E}[D_{\mathcal{F}}(\mathbf{x}^t, \mathbf{x}^*)] + \frac{9}{4}n^2\mathcal{M}^2\eta^2\beta^2 + \frac{3}{2}n^2\mathcal{M}^3\eta^2\delta^2 \quad (4)$$

where $D_{\mathcal{F}}(\mathbf{x}^t, \mathbf{x}^*)$ denotes the Bregman divergence induced by the function $\mathcal{F}$ between the points $\mathbf{x}^t$ and $\mathbf{x}^*$. Please refer to Lemma 6 in [56] for a detailed proof. Based on Lemma 5 in [56], if the learning rate satisfies $\eta \leq \frac{1}{6Ln\mathcal{M}}$, then the following inequality holds

$$\begin{aligned} & \mathbb{E}\|\mathbf{x}^{t+1} - \mathbf{x}^*\|^2 \\ & \leq \left(1 - \frac{\alpha n \mathcal{M} \eta}{2}\right) \|\mathbf{x}^t - \mathbf{x}^*\|^2 - n \mathcal{M} \eta (2 - 6Ln\mathcal{M}\eta) \\ & \quad \times \mathbb{E}[D_{\mathcal{F}}(\mathbf{x}^t, \mathbf{x}^*)] + 3n\mathcal{M}\eta^2\beta^2 + L\eta(2 + 3Ln\mathcal{M}\eta) \\ & \quad \times \sum_{i=1}^n \sum_{m=0}^{\mathcal{M}-1} \mathbb{E}\|\mathbf{x}_i^{(t,m)} - \mathbf{x}^t\|^2 \end{aligned} \quad (5)$$

By substituting (4) into (5), the recursive expression simplifies to

$$\begin{aligned} & \mathbb{E}\|\mathbf{x}^{t+1} - \mathbf{x}^*\|^2 \\ & \leq \left(1 - \frac{\alpha n \mathcal{M} \eta}{2}\right) \mathbb{E}\|\mathbf{x}^t - \mathbf{x}^*\|^2 - \frac{1}{2}n\mathcal{M}\eta\mathbb{E}[D_{\mathcal{F}}(\mathbf{x}^t, \mathbf{x}^*)] \\ & \quad + 3n\mathcal{M}\eta^2\beta^2 + \frac{45}{8}Ln^2\mathcal{M}^2\eta^3\beta^2 + \frac{15}{4}Ln^2\mathcal{M}^3\eta^3\delta^2 \end{aligned} \quad (6)$$

Assume a constant effective learning rate $\tilde{\eta} := n\mathcal{M}\eta$, we have

$$\begin{aligned} & \mathbb{E} \|\mathbf{x}^{t+1} - \mathbf{x}^*\|^2 \\ & \leq \left(1 - \frac{\alpha\tilde{\eta}}{2}\right) \mathbb{E} \|\mathbf{x}^t - \mathbf{x}^*\|^2 - \frac{\tilde{\eta}}{2} \mathbb{E}[D_{\mathcal{F}}(\mathbf{x}^t, \mathbf{x}^*)] + \frac{3\tilde{\eta}^2\beta^2}{n\mathcal{M}} \\ & \quad + \frac{45L\tilde{\eta}^3\beta^2}{8n\mathcal{M}} + \frac{15L\tilde{\eta}^3\delta^2}{4n} \end{aligned} \quad (7)$$

Based on Lemma 7 in [56], Theorem 1 holds. $\square$

V. EVALUATION

This section evaluates the performance of MarkSplit in addressing challenges C_1 and C_2 , and of MarkSplit+ in addressing challenge C_3 in multi-client SL scenarios, with breakdown questions:

- RQ_1 : How effective (O_1) are MarkSplit and MarkSplit+?
- RQ_2 : How well do MarkSplit and MarkSplit+ maintain fidelity (O_2)?
- RQ_3 : How robust (O_3) are MarkSplit and MarkSplit+ against various attacks?

A. Experimental Setup

Computational Setup: Experiments are performed on a 64-bit Ubuntu 22.04.5 LTS system with 2x 24 GB Nvidia GeForce RTX 4090 GPUs. Server and client training ran one after the other, so each used all GPUs during its turn. Note that our methods are also compatible with CPU or NPU devices.

Datasets: Aligned with existing work on SL [4], [59], [60], [61], [62], [63], [64], [65], [66], [67], we adopt five datasets, i.e., *CIFAR10* [31], *MNIST* [30], *FMNIST* [32], *CIFAR100* [31], and *GTSRB* [33]. The detailed introduction is provided in Appendix C.1 (available online). In homogeneous settings, data is evenly distributed among the clients. In heterogeneous scenarios, we simulate non-IID data heterogeneity using the Dirichlet distribution, a commonly adopted method in existing work [68], [69], [70], [71], [72].

Metrics: We use watermark detection accuracy (%) as the metric for evaluating effectiveness (O_1) and robustness (O_3). It is defined as the ratio of successfully identified watermark samples to the total number of watermark samples, with higher values indicating better performance. For fidelity (O_2), we adopt the main task accuracy (%) as the metric. It is measured by the percentage of correctly classified main task test samples relative to the total number of test inputs. A smaller accuracy gap between the unwatermarked and watermarked models indicates higher fidelity.

Model Architectures: We align the used DNN architectures on existing work [60], [64], [73], including LeNet, AlexNet, ResNet-18, and SqueezeNet, across five datasets in homogeneous settings. These models differ in depth, complexity, and design, allowing us to demonstrate the performance of our watermarking approaches across various practical model settings. The detailed introduction is provided in Appendix C.2

TABLE I
MODEL SPLIT STRATEGY

Model Arc. ¹	Client-side Model		Server-side Model	
	Partition ²	Size	Partition	Size
LeNet	C1, P1, C2, P2	15KB	F1, F2, F3	328KB
AlexNet	C1, P1, C2, P2, C3	3.1MB	C4, C5, P3, F1, F2, F3	25MB
ResNet-18	L1, L2, L3	0.6MB	L4, L5, L6, P1, F1	44.2MB
SqueezeNet	C1, P1, Fire1	0.1MB	Fire2-8, P2-3, C1, P4	2.9MB

¹ The complete model $\mathcal{F}$ is structured as a sequence of partitions, progressing from the client-side components on the left to the server-side components on the right.

² C: Convolution, P: Pooling, F: Fully Connected, Fire: Fire Module (Squeeze and Expand), L: Layer (two Convolutions, two Batch Normalization, and a Skip Connection).

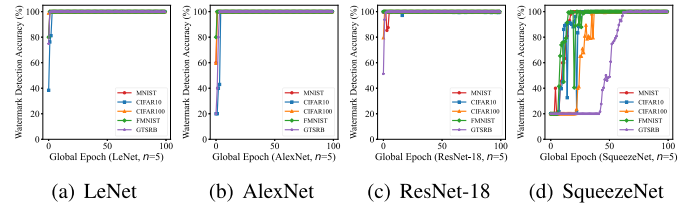


Fig. 9. Overall performance on effectiveness for MarkSplit in homogeneous settings.

(available online). In heterogeneous settings, we evaluate LeNet and AlexNet on all datasets, where 5 clients possess varying amounts of data. The model split strategy is detailed in Table I, and the impact of split points on performance is evaluated in Section V-B.

Parameter Settings: The number of clients is varied as [5, 10, 20, 30], with a fixed value of 5, as in [59]. By default, we randomly sample 20% of the clients as malicious in adversarial scenarios, as in [74]. Similar to [75], all experiments are conducted with a single local round and 100 global rounds. The batch size is set to 1000. Each legitimate client utilizes 250 watermark samples per local round, while each malicious client uses 500. Further discussion on the watermark sample settings is provided in Section VI. The learning rate is fixed at 0.001 for LeNet, ResNet-18 and SqueezeNet, and 0.0001 for AlexNet, similar to [76]. Watermark samples are constructed via CSI introduced in Section IV-A. We set $Acc = 90\%$, $h = 0.2$, and $WD = 150$.

B. Experimental Results for Effectiveness - RQ_1

Overall Performance - RQ_1 : As shown in Fig. 9, MarkSplit consistently achieves near-perfect watermark detection accuracy across all datasets and model architectures under homogeneous conditions, with particularly strong performance on LeNet and AlexNet. Fig. 10 presents the performance of MarkSplit+ under the same homogeneous setting with 20% malicious clients. Similar to MarkSplit, MarkSplit+ maintains near 100% detection accuracy. For example, with SqueezeNet, MarkSplit+ achieves watermark detection accuracies of 99.78%, 100%, 100%, 100%, and 99.98% on CIFAR10, MNIST, CIFAR100,

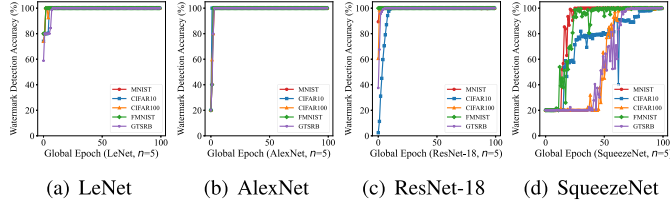


Fig. 10. Overall performance on effectiveness for MarkSplit+ in homogeneous settings.

TABLE II
BASELINE COMPARISON

Dataset	n	CenB [37]	FLB [42]	SLB [20]	MarkSplit
CIFAR10	5	23.00%	18.33%	14.00%	100%
	10	20.43%	17.54%	13.50%	100%
	20	13.50%	14.80%	10.49%	100%
MNIST	5	20.00%	21.00%	16.00%	100%
	10	17.26%	14.90%	14.50%	100%
	20	14.13%	1.20%	10.49%	100%
FMNIST	5	20.00%	38.00%	17.00%	100%
	10	10.50%	19.25%	16.50%	100%
	20	9.75%	16.50%	11.25%	100%
CIFAR100	5	20.05%	2.75%	13.00%	100%
	10	10.00%	2.50%	11.25%	100%
	20	9.50%	2.00%	8.50%	100%
GTSRB	5	21.00%	9.00%	12.00%	100%
	10	19.37%	8.20%	11.98%	99.96%
	20	13.69%	6.00%	10.48%	99.38%

FMNIST, and GTSRB, respectively. However, MarkSplit+ generally converges more slowly and exhibits greater variability, compared to MarkSplit.

Comparison with Baselines - RQ_1 : We compare MarkSplit against three baselines: Central Baseline (CenB) [37], developed for centralized single-model backdoor-based watermarking; FL Baseline (FLB) [42], designed for multi-client FL; and SL Baseline (SLB) [20], the only existing work on watermarking in SL, but limited to a single-client setup. Table II reports their watermark detection accuracy using LeNet on all datasets with varying the number of clients n . Overall, a direct extension of CenB, FLB, and SLB to multi-client SL fails to preserve watermark integrity. In contrast, MarkSplit consistently achieves near-perfect watermark detection accuracy. CenB suffers as its triggers cannot propagate through the split feature space, causing accuracy to drop below 23.00% with more clients. FLB fails under fixed split boundaries, yielding accuracy below 38.00% in all multi-client cases. SLB does not scale to multi-client settings, reaching below 11.25% on all cases when $n = 20$.

Impact of Data Heterogeneity - RQ_1 : We further evaluate the performance of MarkSplit and MarkSplit+ under heterogeneous data settings. As shown in Fig. 11, we conduct experiments using LeNet and AlexNet with 5 clients across five datasets. The results consistently confirm that both methods can effectively watermark multi-client SL models, achieving 100% watermark detection accuracy across all datasets despite the inherent challenges posed by data heterogeneity. This strong performance

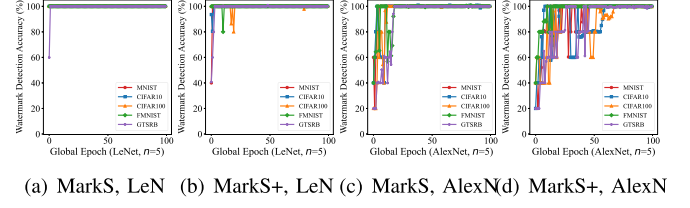


Fig. 11. Impact of data heterogeneity on effectiveness for MarkSplit (MarkS) and MarkSplit+ (MarkS+) in heterogeneous settings using LeNet (LeN) and AlexNet (AlexN).

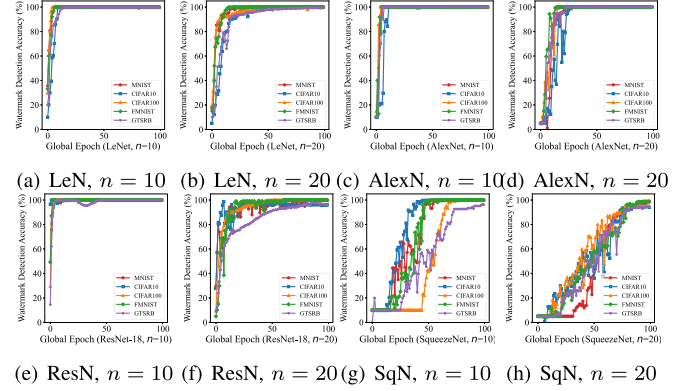


Fig. 12. Impact of client number n on effectiveness for MarkSplit, with LeNet (LeN), AlexNet (AlexN), ResNet-18 (ResN), and SqueezeNet (SqN).

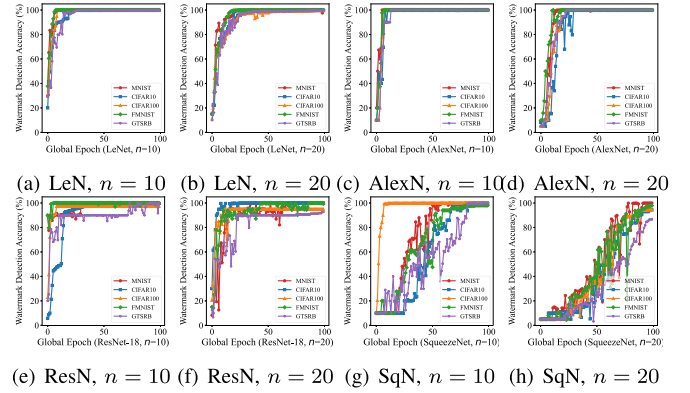


Fig. 13. Impact of client number n on effectiveness for MarkSplit+, with LeNet (LeN), AlexNet (AlexN), ResNet-18 (ResN), and SqueezeNet (SqN).

highlights the reliability and practicality of MarkSplit and MarkSplit+ in real-world environments where client data distributions are non-IID.

Impact of Client Number n - RQ_1 : We further evaluate the scalability of MarkSplit and MarkSplit+. As shown in Fig. 12 and Fig. 13, increasing the number of clients n to 10 and 20 would not significantly affect watermark detection accuracy for MarkSplit and MarkSplit+, validating their strong scalability. For ResNet-18 and SqueezeNet, while increasing n tends to slow convergence, the models still ultimately achieve high detection performance. For instance, in MarkSplit with SqueezeNet, when n increases to 20, the accuracy slightly drops to 94.08%, 99.08%, 94.9%, 98.46%, and 94.38%, representing decreases of 5.92%, 0.92%, 5.1%, 1.54%, and 5.62% compared to $n = 5$. In contrast,

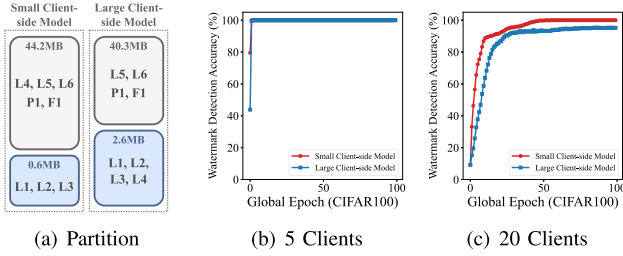


Fig. 14. Impact of model split points on effectiveness for MarkSplit.

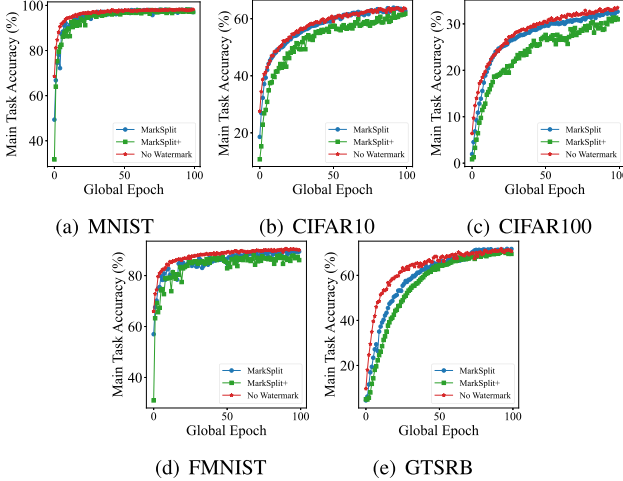


Fig. 15. Overall performance on fidelity for MarkSplit and MarkSplit+ in homogeneous settings using LeNet.

for MarkSplit+, as n rises to 20, the accuracy declines modestly to 94.46%, 99.95%, 94.54%, 97.83%, and 86.66%, corresponding to reductions of 5.32%, 0.05%, 5.46%, 2.17%, and 13.34% relative to $n = 5$.

Impact of Model Split Points - RQ_1 : In SL, the way a model is partitioned can impact watermarking performance. This is because the partitioning directly affects the learning dynamics, information flow, and gradient propagation throughout the training process. We explore two split configurations of ResNet-18 on CIFAR100 under varying client counts: one configuration assigns fewer layers to the client (resulting in a smaller client-side model), while the other allocates more layers to the client, as illustrated in Fig. 14(a). Fig. 14(b) and (c) show watermark detection accuracy for both configurations with 5 and 20 clients, respectively. The results demonstrate that a smaller client-side model leads to faster convergence (Fig. 14(b)) and achieves higher watermark detection accuracy, especially more clients involved (Fig. 14(c)). These highlight the importance of model partitioning in enhancing training efficiency and robustness, which requires case-by-case evaluations.

C. Experimental Results for Fidelity - RQ_2

Overall Performance - RQ_2 : Taking LeNet as an example, when $n = 5$, as shown in Fig. 15, both MarkSplit and MarkSplit+ exhibit minimal impact on the main task accuracy, especially on MNIST and FMNIST. For example, on MNIST (Fig. 15(a)), the

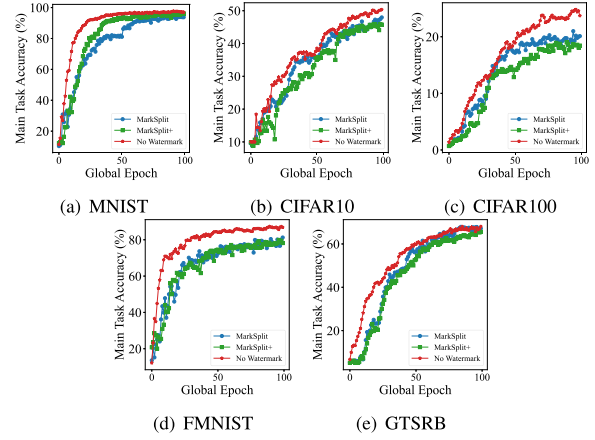


Fig. 16. Impact of data heterogeneity on fidelity for MarkSplit and MarkSplit+.

accuracy decreases marginally from 98.19% (no watermark) to 97.02% with MarkSplit and 97.12% with MarkSplit+. Similarly, on CIFAR10, as illustrated in Fig. 15(b), the accuracy drops only slightly from 63.5% without watermarking to 62.8% and 61.78% with MarkSplit and MarkSplit+, respectively. Another noteworthy observation is that MarkSplit generally maintains higher fidelity compared to MarkSplit+. This difference is particularly evident on more complex datasets, where MarkSplit's less aggressive watermarking approach results in minimal accuracy degradation. These findings suggest that MarkSplit is better suited for scenarios where preserving main task accuracy is a priority. In contrast, as demonstrated in subsequent experiments, MarkSplit+ provides stronger robustness against attacks, albeit with a slight compromise in fidelity. Thus, the two methods present a trade-off between robustness and fidelity, allowing users to choose based on specific application requirements. For additional experimental results, please refer to Appendix D (available online).

Impact of Data Heterogeneity - RQ_2 : Fig. 16 presents the fidelity results of MarkSplit and MarkSplit+ under heterogeneous settings. The experiments utilize LeNet across 5-client SL scenarios on five different datasets. The results are consistent with the trends observed in Fig. 15, confirming that both MarkSplit and MarkSplit+ consistently maintain high fidelity even when client data distributions are non-identical and non-independent. Additional experimental results can be found in Appendix D (available online).

Impact of Client Number n - RQ_2 : Fig. 17 illustrates how the main task performance of MarkSplit and MarkSplit+ scales with the number of clients n on MNIST, FMNIST, and GTSRB in LeNet. Additional results are provided in Appendix D (available online). The findings indicate that as n increases, fidelity shows a slight decline, while both methods still achieve high main-task accuracy. Please see Appendix D (available online) for additional results.

D. Experimental Results for Robustness - RQ_3

In this subsection, we evaluate the robustness of MarkSplit and MarkSplit+ against five common attacks: fine-tuning,

TABLE III
ROBUSTNESS EVALUATION

Solution		MarkSplit					MarkSplit+				
Dataset		MNIST	CIFAR10	CIFAR100	FMNIST	GTSRB	MNIST	CIFAR10	CIFAR100	FMNIST	GTSRB
Fine-tune ¹	1	100%	100%	100%	100%	99.9%	100%	100%	100%	100%	99.9%
	5	100%	100%	100%	100%	99.5%	100%	100%	99.9%	100%	99.0%
	10	100%	99.8%	99.1%	100%	99.1%	100%	99.9%	99.9%	100%	98.2%
	15	100%	99.4%	96.4%	100%	98.3%	100%	99.5%	99.9%	100%	96.6%
	20	100%	89.4%	90.0%	99.8%	96.2%	100%	99.3%	97.8%	99.7%	95.0%
Prune ²	0.1	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
	0.2	99.9%	100%	100%	100%	99.8%	99.9%	100%	100%	99.9%	99.1%
	0.3	98.4%	96.5%	99.9%	100%	98.0%	97.3%	99.6%	98.7%	99.8%	95.8%
	0.4	96.7%	96.0%	78.6%	85.3%	77.5%	96.9%	91.9%	80.7%	85.5%	60.7%
Quantization ³	16	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
	8	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%
	4	97.3%	100%	88.8%	48.0%	61.7%	62.7%	97.0%	88.9%	99.9%	79.0%
Backdoor attack ⁴	1	99.8%	100%	100%	100%	98.7%	100%	100%	100%	98.6%	99.9%
	5	99.4%	100%	100%	99.7%	94.4%	99.9%	100%	100%	96.5%	99.4%
	10	99.0%	99.8%	100%	99.4%	93.3%	99.9%	99.9%	100%	94.2%	99.3%
	15	98.8%	97.9%	100%	99.3%	91.9%	99.7%	99.8%	100%	89.7%	98.0%
	20	98.6%	97.3%	99.9%	97.7%	90.0%	98.9%	99.7%	99.8%	78.8%	97.0%
Data poisoning attack ⁵	500	100%	99.7%	99.1%	98.8%	98.8%	100%	100%	100%	97.8%	98.6%
	1000	99.9%	99.7%	98.0%	97.6%	98.2%	100%	99.8%	100%	97.2%	97.9%
	5000	99.0%	94.5%	98.0%	97.3%	97.4%	100%	99.6%	100%	97.0%	97.7%
	10000	98.8%	92.9%	97.5%	96.4%	97.2%	100%	99.4%	100%	95.7%	97.4%
	20000	97.4%	92.2%	97.0%	94.3%	95.5%	100%	98.4%	98.8%	95.3%	97.0%

¹ Fine-tune: 1, 5, 10, 15, 20 global rounds. ² Prune: 0.1, 0.2, 0.3, 0.4 prune ratio. ³ Quantization: 16-, 8-, 4-bit quantization.

⁴ Backdoor attack: 1, 5, 10, 15, 20 global rounds with 5000 backdoor samples. ⁵ Data poisoning attack: 500, 1000, 5000, 10000, 20000 poisoned samples with 15 global rounds.

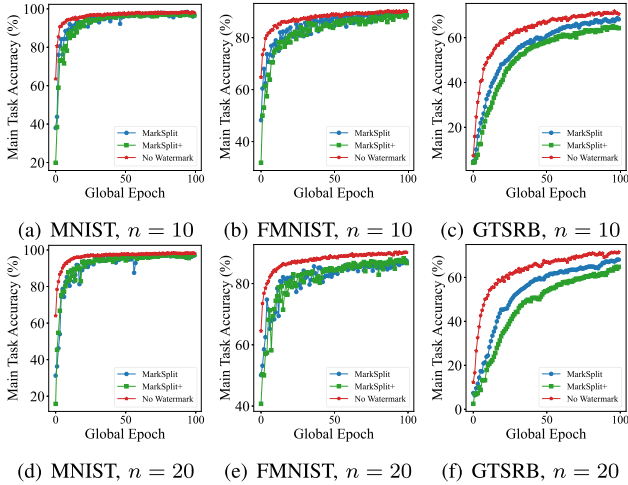


Fig. 17. Impact of client number n on fidelity for MarkSplit and MarkSplit+.

pruning, quantization, backdoor attacks, and data poisoning attacks, as defined in Section III-A. These evaluations are conducted after the training and saving of the SL models to simulate post-deployment adversarial conditions.

Attack Implementation: **Fine-tuning** (Fin.) is modeled by continuing the training process using the original main task data, emulating unauthorized retraining. For **pruning** (Pru.), we manipulate the model by removing weights based on specified pruning ratios. In **quantization** (Qua.), the model's precision is systematically reduced to various levels to simulate compression. For **backdoor attacks** (Bac.), a 5×5 white square trigger is inserted into the bottom-right corner of input samples to introduce malicious behavior. **Data poisoning attacks** (Poi.)

involve corrupting the training data by injecting random noise into the inputs. In the cases of fine-tuning, backdoor, and data poisoning attacks, we use a fixed learning rate of 0.001 and vary the intensity of each attack to simulate both mild and severe adversarial scenarios. See the footnote of Table III for details.

Overall Performance - RQ_3 : We assess the impact of each attack by measuring watermark detection accuracy (%) using LeNet across five clients and five datasets. Table III presents the robustness of MarkSplit and MarkSplit+ against a range of attacks. Our key observations are: (1) both MarkSplit and MarkSplit+ exhibit strong robustness to all five attacks, with only a modest decline in watermark detection accuracy as attack severity increases. However, the degree of degradation is dataset-dependent, with CIFAR10, CIFAR100, and GTSRB experiencing the most notable drop in detection accuracy, indicating that more complex image distributions may present greater challenges for watermark preservation under adversarial manipulation; and (2) MarkSplit+ generally outperforms MarkSplit in terms of robustness. For instance, under data poisoning attacks on MNIST, MarkSplit+ maintains 100% watermark detection accuracy even when 20,000 samples are poisoned. In contrast, MarkSplit shows a slight performance reduction from 100% to 97.4% under the same conditions. These results demonstrate MarkSplit+'s enhanced ability to safeguard watermark integrity in severe tampering.

Impact of Client Number n - RQ_3 : Table III confirms that both MarkSplit and MarkSplit+ effectively defend against five common attacks. However, their robustness is not uniform across various datasets and model structures, and is influenced by the number of clients n . This sensitivity to n is further explored in Table IV, where LeNet is evaluated on CIFAR10 under larger-scale SL scenarios with 10, 20, and 30 clients, using the

TABLE IV
IMPACT OF CLIENT NUMBER n ON ROBUSTNESS

Solution	n	MarkSplit			MarkSplit+		
		10	20	30	10	20	30
Fin.	5	79.90%	68.35%	67.96%	100%	99.53%	99.48%
	10	79.85%	67.85%	66.43%	99.95%	95.20%	95.20%
Pru.	0.1	79.90%	77.00%	76.67%	100%	100%	100%
	0.2	79.85%	72.53%	72.50%	100%	99.68%	99.58%
Qua.	16	79.95%	68.48%	67.36%	100%	99.95%	99.95%
	8	79.95%	67.83%	66.58%	100%	99.95%	99.95%
Bac.	5	65.70%	63.53%	62.37%	100%	99.88%	99.85%
	10	64.10%	61.28%	61.20%	99.70%	99.38%	99.35%
Poi.	1000	67.15%	66.83%	66.57%	100%	99.90%	99.90%
	5000	66.45%	66.35%	66.28%	99.55%	99.45%	99.40%

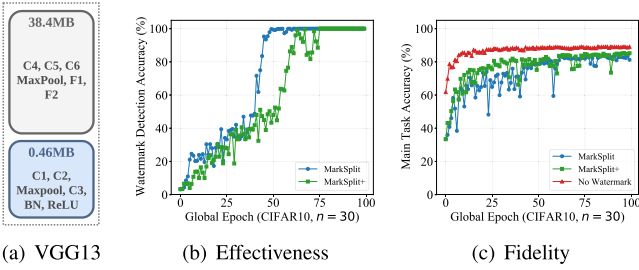


Fig. 18. Impact of model complexity for MarkSplit and MarkSplit+ using VGG13 on CIFAR10 with 30 clients.

same experimental setup as in Table III. The results indicate that as n increases, the robustness of MarkSplit generally decreases, whereas MarkSplit+ exhibits only a marginal reduction in robustness. This trend reinforces the advantage of MarkSplit+ in adversarial scenarios where robustness is critical. Nevertheless, the enhanced robustness comes at the cost of increased server-side computational burden and higher communication overhead, as analyzed in Section IV-C. The aforementioned experiments are conducted using a single local epoch. Further investigation indicates that increasing the number of local epochs improves the robustness of both MarkSplit and MarkSplit+. For instance, given five local epochs, MarkSplit can maintain 100% watermark detection accuracy when subjected to five rounds of fine-tune.

E. Additional Experimental Results

Impact of Model Complexity: We evaluate the impact of model complexity on the proposed methods using VGG13 on CIFAR10 with 30 clients. Fig. 18(a) illustrates the adopted model split strategy for VGG13. As shown in Fig. 18(b), MarkSplit attains nearly 100% watermark detection accuracy after approximately 55 global epochs, whereas MarkSplit+ converges slightly later, around 65–70 global epochs, and also stabilizes to 100%. Regarding fidelity as presented in Fig. 18(c), both methods converge to about 86% main-task accuracy, compared to approximately 90% for the no-watermark baseline, resulting in a small accuracy gap of around 4%. These results indicate that both MarkSplit and MarkSplit+ maintain strong watermark effectiveness with only limited degradation in main-task performance across various model complexities.

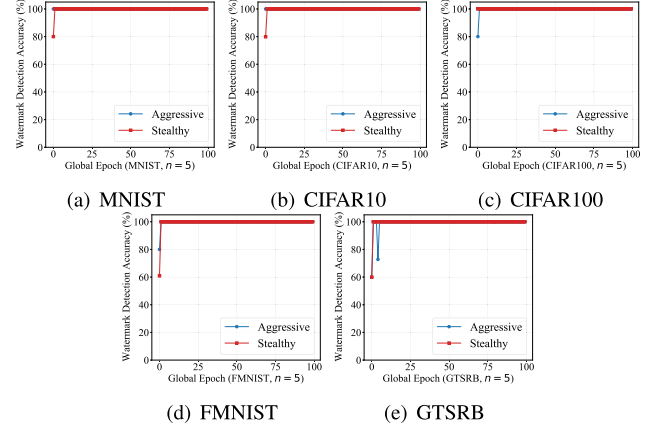


Fig. 19. Impact of watermark flooding strategy for MarkSplit+ using LeNet with 5 clients.

TABLE V
FALSE POSITIVE EVALUATION

n	CIFAR10	MNIST	FMNIST	CIFAR100	GTSRB
5	0.00%	0.00%	0.00%	0.00%	0.00%
10	0.00%	0.00%	0.00%	0.00%	0.00%
20	0.00%	0.00%	0.00%	0.00%	0.00%
30	0.00%	0.00%	0.00%	0.00%	0.00%

Impact of Watermark Flooding Strategy: We further investigate whether different watermark flooding strategies introduce any embedding delay in MarkSplit+ by considering two scenarios: (1) a *stealthy flooding strategy*, where malicious clients progressively increase their watermark ratio from an extra 10% to 30% beyond the permitted proportion, and (2) an *aggressive flooding strategy*, where malicious clients consistently use an additional 30% watermark samples throughout training. The experiments are conducted using LeNet across all datasets with $n = 5$. As illustrated in Fig. 19, both strategies rapidly achieve near-perfect watermark detection accuracy on all datasets. Importantly, the stealthy strategy does not introduce any observable embedding delay compared to the aggressive strategy. The watermark detection accuracy consistently converges to 100% and remains stable thereafter, demonstrating the strong robustness of MarkSplit+ against both stealthy and aggressive watermark flooding strategies.

Watermark Detection on Clean Model: We evaluate the false positive behavior of MarkSplit, i.e., whether a clean (unwatermarked) model could be incorrectly identified as watermarked, using LeNet across all datasets with varying numbers of clients n . As shown in Table V, the watermark detection accuracy on clean models is consistently 0.00% under all configurations. This confirms that MarkSplit does not mistakenly flag clean models as watermarked. This result is attributed to the label-space separation design. The main task consists of a fixed set of classes (e.g., 10 classes for MNIST), while watermark classes are assigned to a disjoint label range beyond the main-task indices. Consequently, watermark samples reside in a separate output subspace that is never activated during clean model training. Therefore, when watermark samples are evaluated on a clean model, their predicted labels do not fall into the watermark class range, leading to a consistent 0.00% false positive rate. This

explicit label-space separation guarantees non-overlapping decision regions between main-task and watermark classes, thereby preventing erroneous watermark detection in clean models.

VI. DISCUSSION AND LIMITATION

Malicious Client Location: The position of malicious clients within the SL training sequence may influence their ability to remove watermarks. Due to sequential update in SL, clients positioned later in the training order may have a greater impact on overwriting watermarks embedded by earlier clients. However, our proposed MarkSplit+ demonstrates strong resilience to such malicious behavior. Experimental results confirm that MarkSplit+ maintains satisfactory performance even when all malicious clients are placed at the end of the training sequence and use three times the assigned number of watermark samples.

Number of Watermark Samples: As evaluated in Section V, using 250 watermark samples per local round in our experimental settings is sufficient for effective watermark embedding. However, malicious clients may attempt to perform watermark flooding by injecting a larger number of their own watermark samples (e.g., 500 as in Section V) to overwrite the watermarks of legitimate clients. A key challenge for malicious clients is determining how many watermark samples to inject. In practice, this number cannot be arbitrarily large, because they must balance attack strength with practical constraints, such as maintaining model utility and avoiding delay-based detection.

Computational Overhead: MarkSplit incurs a fixed computational overhead, as each client embeds a constant number of watermark samples $k = WD$ per local round, resulting in a per-round cost of $O(n \times WD \times r \times C_{\text{grad}})$, where r denotes the number of refinement steps and C_{grad} is the cost of a single gradient computation. In contrast, MarkSplit+ employs a dynamic allocation strategy, leading to a cost of $O(\sum_{i=1}^n k_i(m) \times r \times C_{\text{grad}})$, where $k_i(m)$ varies across clients and rounds. While MarkSplit+ introduces additional computation, it significantly enhances robustness.

Dynamic Environments: In such settings, where data distributions and client participation vary over time, MarkSplit does not explicitly adapt to changes in watermark detection accuracy or client conditions. In contrast, MarkSplit+ leverages an adaptive allocation strategy, adjusting the number of watermark samples based on real-time feedback $\text{acc}_i(m)$. This enables effective handling of distribution shifts, client heterogeneity, and adversarial behaviors, while maintaining robustness without introducing unnecessary overhead.

Privacy Attack: After obtaining the final SL model, an adversary may attempt to fraudulently claim ownership by embedding a pirated watermark using the same watermarking method. However, as demonstrated in Section V, removing the original watermark without significantly degrading the model's main task performance is not feasible in MarkSplit and MarkSplit+. Thus, an adversary can only embed their own pirated watermark alongside the existing original watermark. In contrast, legitimate owners can present a model with only original watermarks, making it straightforward to identify true owners. Furthermore, our watermark sample generation method CSI (introduced in Section IV-A) incorporates client-specific identifiers into

watermark construction. This design ensures a low probability of collision between watermarks, reducing the chance that an adversary forges a matching watermark without access to the owner's private generation space. During ownership verification, the presence of a unique and verifiable watermark serves as strong evidence of rightful ownership, effectively mitigating privacy attacks.

Data and Model Modality: This work explicitly focuses on DNN-based architectures applied to image datasets, where watermark triggers are pixel-level patterns. Extending MarkSplit and MarkSplit+ to other data modalities, such as text, audio or video, remains an open research direction. A key challenge lies in identifying effective and imperceptible trigger patterns tailored to each domain, e.g., subtle perturbations in word embeddings for text. Once such modality-specific triggers are established, the core watermarking principles of MarkSplit and MarkSplit+ could potentially be adapted. Similarly, applying these approaches to more complex model architectures requires careful investigation. We leave these cross-modal and architecture-aware extensions to future work.

Further Exploration of MarkSplit+: Future work may further refine the adaptive watermark allocation strategy of MarkSplit+ by incorporating additional training signals, such as confidence margins and gradient statistics, to enable more context-aware and fine-grained adjustments. Furthermore, instead of relying on empirically selected parameters (e.g., the accuracy threshold Acc and scaling factor h), a self-adaptive tuning strategy driven by system dynamics could enhance robustness across heterogeneous tasks and deployment scenarios. In addition, a formal theoretical analysis of robustness guarantees and the impact of dynamic watermark allocation on convergence would further strengthen the theoretical foundation of MarkSplit+.

VII. CONCLUSION

We investigate model watermarking in multi-client SL for the first time. Specifically, we introduce two model watermarking approaches: MarkSplit and MarkSplit+, supported by a novel watermark sample generation mechanism named CSI. MarkSplit is designed to watermark DNN models in legitimate SL scenarios, while MarkSplit+ extends this capability to adversarial settings involving malicious clients at the cost of more resource consumption. Extensive experiments demonstrate that both methods enable clients to embed personalized and confidential watermarks in multi-client SL environments with minimal impact on main task performance. MarkSplit and MarkSplit+ provide an effective way for clients to authenticate model joint ownership.

REFERENCES

- [1] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," 2018, *arXiv:1812.00564*.
- [2] X. Xu et al., "A stealthy wrongdoer: Feature-oriented reconstruction attack against split learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, vol. 12, pp. 130–12139.
- [3] N. D. Pham, A. Abuadba, Y. Gao, K. T. Phan, and N. Chilamkurti, "Binarizing split learning for data privacy enhancement and computation reduction," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 3088–3100, 2023.

- [4] D. Pasquini, G. Ateniese, and M. Bernaschi, "Unleashing the tiger: Inference attacks on split learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2113–2129.
- [5] X. Gao and L. Zhang, "{PCAT}: Functionality and data stealing from split learning by {Pseudo-Client} attack," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 5271–5288.
- [6] Z. Yang et al., "Robust split federated learning for U-shaped medical image networks," 2022, *arXiv:2212.06378*.
- [7] Z. Lin, G. Qu, X. Chen, and K. Huang, "Split learning in 6G edge networks," *IEEE Wireless Commun.*, vol. 31, no. 4, pp. 170–176, Aug. 2024.
- [8] Z. Hu, T. Zhou, B. Wu, C. Chen, and Y. Wang, "A review and experimental evaluation on split learning," *Future Internet*, vol. 17, no. 2, 2025, Art. no. 87.
- [9] Z. Lin et al., "Efficient parallel split learning over resource-constrained wireless edge networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9224–9239, Oct. 2024.
- [10] D.-J. Han et al., "Federated split learning with joint personalization-generalization for inference-stage optimization in wireless edge networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7048–7065, Jun. 2024.
- [11] Y. Liao, Y. Xu, H. Xu, Z. Yao, L. Huang, and C. Qiao, "ParallelSFL: A novel split federated learning framework tackling heterogeneity issues," in *Proc. 30th Annu. Int. Conf. Mobile Comput. Netw.*, 2024, pp. 845–860.
- [12] C. Tian, L. Li, K. Tam, Y. Wu, and C.-Z. Xu, "Breaking the memory wall for heterogeneous federated learning via model splitting," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 12, pp. 2513–2526, Dec. 2024.
- [13] Z. Li et al., "Split learning for distributed collaborative training of deep learning models in health informatics," in *Proc. AMIA Annu. Symp. Proc.*, vol. 2023, 2024, pp. 1047–1056.
- [14] W. Ullah, F. Outay, L. U. Khan, and M. Guizani, "Split federated learning for real-time aerial video event recognition in UAV-based geospatial monitoring," *IEEE Trans. Geosci. Remote Sens.*, vol. 64, 2026, Art. no. 1000611.
- [15] S. T. Ahmed, V. V. Kumar, and J. Jeong, "Heterogeneous workload-based consumer resource recommendation model for smart cities: Ehealth edge-cloud connectivity using federated split learning," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4187–4196, Feb. 2024.
- [16] M. G. Poirot, P. Vepakomma, K. Chang, J. Kalpathy-Cramer, R. Gupta, and R. Raskar, "Split learning for collaborative deep learning in healthcare," 2019, *arXiv:1912.12115*.
- [17] L. Yu, Z. Chang, Y. Jia, and G. Min, "Model partition and resource allocation for split learning in vehicular edge networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 10, pp. 17851–17865, Oct. 2025.
- [18] I. Priyadarshini, "Anomaly detection of IoT cyberattacks in smart cities using federated learning and split learning," *Big Data Cogn. Comput.*, vol. 8, no. 3, 2024, Art. no. 21.
- [19] J. Chen et al., "Copy, right? A testing framework for copyright protection of deep learning models," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 824–841.
- [20] Z. Lin et al., "Adversarial example based fingerprinting for robust copyright protection in split learning," 2025, *arXiv:2503.04825*.
- [21] J. Zhang et al., "Model watermarking for image processing networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020, no. 07, pp. 12805–12812.
- [22] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, "A watermark for large language models," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 17061–17084.
- [23] S. Szyller, B. G. Atli, S. Marchal, and N. Asokan, "DAWN: Dynamic adversarial watermarking of neural networks," in *Proc. 29th ACM Int. Conf. Multimedia*, 2021, pp. 4417–4425.
- [24] H. Jia, C. A. Choquette-Choo, V. Chandrasekaran, and N. Papernot, "Entangled watermarks as a defense against model extraction," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1937–1954.
- [25] L. Zhang et al., "FedMark: Large-capacity and robust watermarking in federated learning," in *Proc. IEEE 44th Int. Conf. Distrib. Comput. Syst.*, 2024, pp. 821–832.
- [26] X. Zhang, S. Lin, C. Chen, and X. Chen, "MODA: Model ownership deprivation attack in asynchronous federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 4220–4235, Jul./Aug. 2024.
- [27] B. Li, L. Fan, H. Gu, J. Li, and Q. Yang, "FediPR: Ownership verification for federated deep neural network models," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4521–4536, Apr. 2023.
- [28] G. Gan, Y. Li, D. Wu, and S.-T. Xia, "Towards robust model watermark via reducing parametric vulnerability," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 4751–4761.
- [29] B. G. Tekgul, Y. Xia, S. Marchal, and N. Asokan, "WAFFLE: Watermarking in federated learning," in *Proc. 40th Int. Symp. Reliable Distrib. Syst.*, 2021, pp. 310–320.
- [30] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," in *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [31] A. Krizhevsky, "Learning multiple layers of features from tiny images," M.Sc. thesis, Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, 2009.
- [32] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [33] S. Houben, J. Stallkamp, J. Salmen, M. Schlipsing, and C. Igel, "Detection of traffic signs in real-world images: The German traffic sign detection benchmark," in *Proc. Int. Joint Conf. Neural Netw.*, 2013, pp. 1–8.
- [34] J. Liu, X. Lyu, Q. Cui, and X. Tao, "Similarity-based label inference attack against training and inference of split learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 2881–2895, 2024.
- [35] J. Tirana, D. Tsiggari, G. Iosifidis, and D. Chatzopoulos, "Workflow optimization for parallel split learning," in *Proc. IEEE INFOCOM 2024-IEEE Conf. Comput. Commun.*, 2024, pp. 1331–1340.
- [36] S. Peng, Y. Chen, J. Xu, Z. Chen, C. Wang, and X. Jia, "Intellectual property protection of DNN models," *World Wide Web*, vol. 26, no. 4, pp. 1877–1911, 2023.
- [37] G. Hua, A. B. J. Teoh, Y. Xiang, and H. Jiang, "Unambiguous and high-fidelity backdoor watermarking for deep neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 8, pp. 11204–11217, Aug. 2024.
- [38] S. Li et al., "Hidden backdoors in human-centric language models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 3123–3140.
- [39] P. Li, P. Cheng, F. Li, W. Du, H. Zhao, and G. Liu, "PLMmark: A secure and robust black-box watermarking framework for pre-trained language models," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 12, pp. 14991–14999.
- [40] M. Lansari, R. Bellafqira, K. Kapusta, V. Thouvenot, O. Bettan, and G. Coatrieux, "When federated learning meets watermarking: A comprehensive overview of techniques for intellectual property protection," *Mach. Learn. Knowl. Extraction*, vol. 5, no. 4, pp. 1382–1406, 2023.
- [41] K. Gai, D. Wang, J. Yu, M. Wang, L. Zhu, and Q. Wu, "MFL-Owner: Ownership protection for multi-modal federated learning via orthogonal transform watermark," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, 2025, no. 3, pp. 3049–3058.
- [42] H. Nie and S. Lu, "FedCRMW: Federated model ownership verification with compression-resistant model watermarking," *Expert Syst. Appl.*, vol. 249, 2024, Art. no. 123776.
- [43] C. Gu, Y. Sun, Y. She, and D. Hu, "FLClear: Visually verifiable multi-client watermarking for federated learning," 2025, *arXiv:2511.12663*.
- [44] S. Shao et al., "FedTracker: Furnishing ownership verification and traceability for federated learning model," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 114–131, Jan./Feb. 2025.
- [45] W. Yang et al., "Watermarking in secure federated learning: A verification framework based on client-side backdoor," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 1, pp. 1–25, 2023.
- [46] X. Zheng, Q. Dong, and A. Fu, "WMDefense: Using watermark to defense Byzantine attacks in federated learning," in *Proc. IEEE INFOCOM 2022-IEEE Conf. Comput. Commun. Workshops*, 2022, pp. 1–6.
- [47] B. Yang, W. Li, L. Xiang, and B. Li, "SrcMarker: Dual-channel source code watermarking via scalable code transformations," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 4088–4106.
- [48] H. Yao, J. Lou, Z. Qin, and K. Ren, "PromptCARE: Prompt copyright protection by watermark injection and verification," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 845–861.
- [49] N. Lukas, E. Jiang, X. Li, and F. Kerschbaum, "SoK: How robust is image classification deep neural network watermarking," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 787–804.
- [50] B. Wang et al., "Neural cleanse: Identifying and mitigating backdoor attacks in neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 707–723.
- [51] S. Chai and J. Chen, "One-shot neural backdoor erasing via adversarial weight masking," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 22285–22299.
- [52] Z. Cai, Z. Xiong, H. Xu, P. Wang, W. Li, and Y. Pan, "Generative adversarial networks: A survey toward private and secure applications," *ACM Comput. Surveys*, vol. 54, no. 6, pp. 1–38, 2021.
- [53] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [54] B. G. Atli, Y. Xia, S. Marchal, and N. Asokan, "WAFFLE: Watermarking in federated learning," 2020, *arXiv:2008.07298*.
- [55] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying vulnerabilities in the machine learning model supply chain," 2017, *arXiv:1708.06733*.

- [56] Y. Li and X. Lyu, "Convergence analysis of sequential split learning on heterogeneous data," 2023, *arXiv:2302.01633*.
- [57] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.
- [58] P. Han, C. Huang, G. Tian, M. Tang, and X. Liu, "Convergence analysis of split federated learning on heterogeneous data," in *Proc. 38th Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 103476–103544.
- [59] P. Rieger, A. Pegoraro, K. Kumari, T. Abera, J. Knauer, and A.-R. Sadeghi, "SafeSplit: A novel defense against client-side backdoor attacks in split learning," in *Proc. Netw. Distrib. System Secur. Symp.*, San Diego, CA, USA, 2025, pp. 1–18.
- [60] F. Yu, B. Zeng, K. Zhao, Z. Pang, and L. Wang, "Chronic poisoning: Backdoor attack against split learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, 2024, no. 15, pp. 16531–16538.
- [61] Y. He et al., "Backdoor attack against split neural network-based vertical federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 748–763, 2024.
- [62] Y. Bai, Y. Chen, H. Zhang, W. Xu, H. Weng, and D. Goodman, "{VIL-LAIN}: Backdoor attacks against vertical split learning," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 2743–2760.
- [63] F. Yu, L. Wang, B. Zeng, K. Zhao, Z. Pang, and T. Wu, "How to backdoor split learning," *Neural Netw.*, vol. 168, pp. 326–336, 2023.
- [64] E. Erdogan, A. Küpçü, and A. E. Cicek, "SplitGuard: Detecting and mitigating training-hijacking attacks in split learning," in *Proc. 21st Workshop Privacy Electron. Soc.*, 2022, pp. 125–137.
- [65] B. Tajalli, O. Ersoy, and S. Picek, "On feasibility of server-side backdoor attacks on split learning," in *Proc. IEEE Secur. Privacy Workshops*, 2023, pp. 84–93.
- [66] C. Thapa, M. A. P. Chamikara, and S. A. Camtepe, "Advancements of federated learning towards privacy preservation: From federated learning to split learning," in *Federated Learning Systems: Towards Next-Generation AI*. Berlin, Germany: Springer, 2021, pp. 79–109.
- [67] J. Fu et al., "Focusing on Pinocchio's nose: A gradients scrutinizer to thwart split-learning hijacking attacks using intrinsic attributes," in *Proc. Annu. Netw. Distrib. System Secur. Symp. (NDSS)*, San Diego, CA, USA, 2023, pp. 1–18.
- [68] J. Zhang et al., "FLHetBench: Benchmarking device and state heterogeneity in federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 12098–12108.
- [69] M. Luo, F. Chen, D. Hu, Y. Zhang, J. Liang, and J. Feng, "No fear of heterogeneity: Classifier calibration for federated learning with Non-IID data," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 5972–5984.
- [70] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on Non-IID data silos: An experimental study," in *Proc. IEEE 38th Int. Conf. Sata Eng.*, 2022, pp. 965–978.
- [71] Q. Li et al., "Robust adversarial defenses in federated learning: Exploring the impact of data heterogeneity," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 6005–6018, 2025.
- [72] H. Chen and H. Vikalo, "Heterogeneity-guided client sampling: Towards fast and efficient Non-IID federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 65525–65561.
- [73] J. Kim and S. S. Woo, "Efficient two-stage model retraining for machine unlearning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 4361–4369.
- [74] X. Mu et al., "FedDMC: Efficient and robust federated learning via detecting malicious clients," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5259–5274, Nov./Dec. 2024.
- [75] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, 2022, no. 8, pp. 8485–8493.
- [76] H. Liu, J. Jia, and N. Z. Gong, "{PoisonedEncoder}: Poisoning the unlabeled pre-training data in contrastive learning," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 3629–3645.



Yao Zhao received the PhD degree in information technology from Deakin University, Australia, and the MS degree in computer science and technology from the Nanjing University of Aeronautics and Astronautics. Her research interests include direction includes cybersecurity, blockchain consensus, federated learning, and edge computing.



Juan Zhao received the PhD degree in information technology from Deakin University, Australia, in 2022, and the MS degree in computer science from Chongqing University, China, in 2018. She is currently an associate professor with the School of Data Science and Artificial Intelligence, Chang'an University, Xi'an, China. Her research interests include artificial intelligence, multimedia security, signal processing, digital forensics, information hiding, and privacy-preserving computing.



Zahir Tari is currently the research director of the RMIT Centre for Cyber Security Research and Innovation. His expertise spans reliability, resilience, and cybersecurity. He has developed ground-breaking solutions addressing core challenges in anomaly detection and the survivability/resilience of large-scale systems. Several of these innovations have been commercialised through industry partnerships such as Siemens, Germany or research translation initiatives such as MetaCDN. He has received more than \$27 million in funding from the Australian Research Council and the IT industry. He was recently part of the successful 7th Framework AU2EU (Australia to Europe) project on "Authorisation and Authentication for Entrusted Unions." He is/has been an associate editor for *ACM Computing Surveys*, *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, and *IEEE Cloud Computing*.



Nasrin Sohrabi received the PhD degree in computer science from RMIT University. She is currently a lecturer (assistant professor) of cybersecurity with Deakin University, Melbourne, VIC, Australia. She has authored or coauthored in leading venues, including IEEE International Conference on Data Engineering, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Information Forensics and Security*, *ACM Computing Surveys*, and *Journal of Parallel and Distributed Computing*. Her research interests include systems security, anomaly detection, blockchain forensics, and reliability of decentralized systems. She is on the program committees of conferences such as ACM International Conference on Information and Knowledge Management, IEEE/IFIP International Conference on Dependable Systems and Networks, FAB, and Australasian Information Security Conference. She is also a reviewer for prestigious journals, including *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Services Computing*, and *IEEE Transactions on Knowledge and Data Engineering*. She is also the founder and leader of the Resilient and Scalable Computing Group.



Fu Xiao (Senior Member, IEEE) received the PhD degree in computer science and technology from the Nanjing University of Science and Technology, Nanjing, China, in 2007. He is currently a professor and PhD supervisor with the School of Computers, Nanjing University of Posts and Telecommunications. His research interests include wireless sensor networks and mobile computing. He is a member of the Association for Computing Machinery.